

Week 12 Lecture

Welcome. This week you will learn:

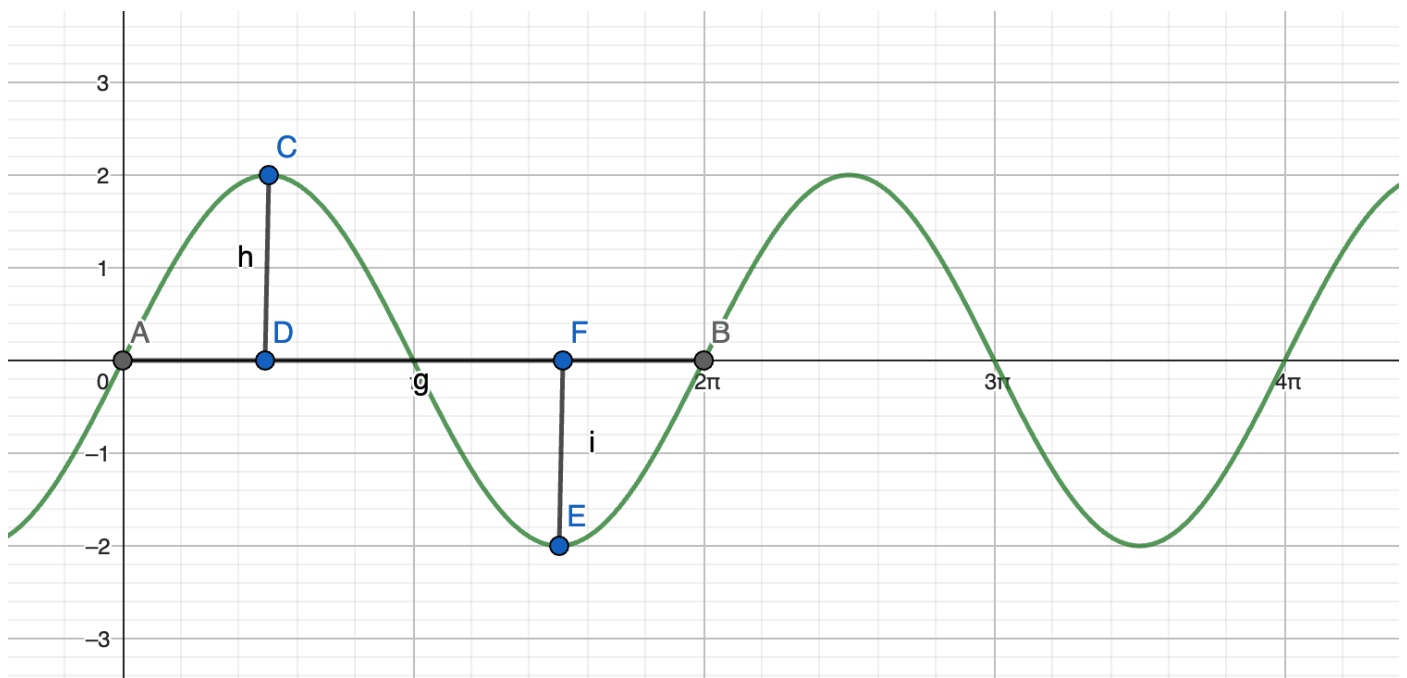
- Sound in p5.js

1. Basics of Sound

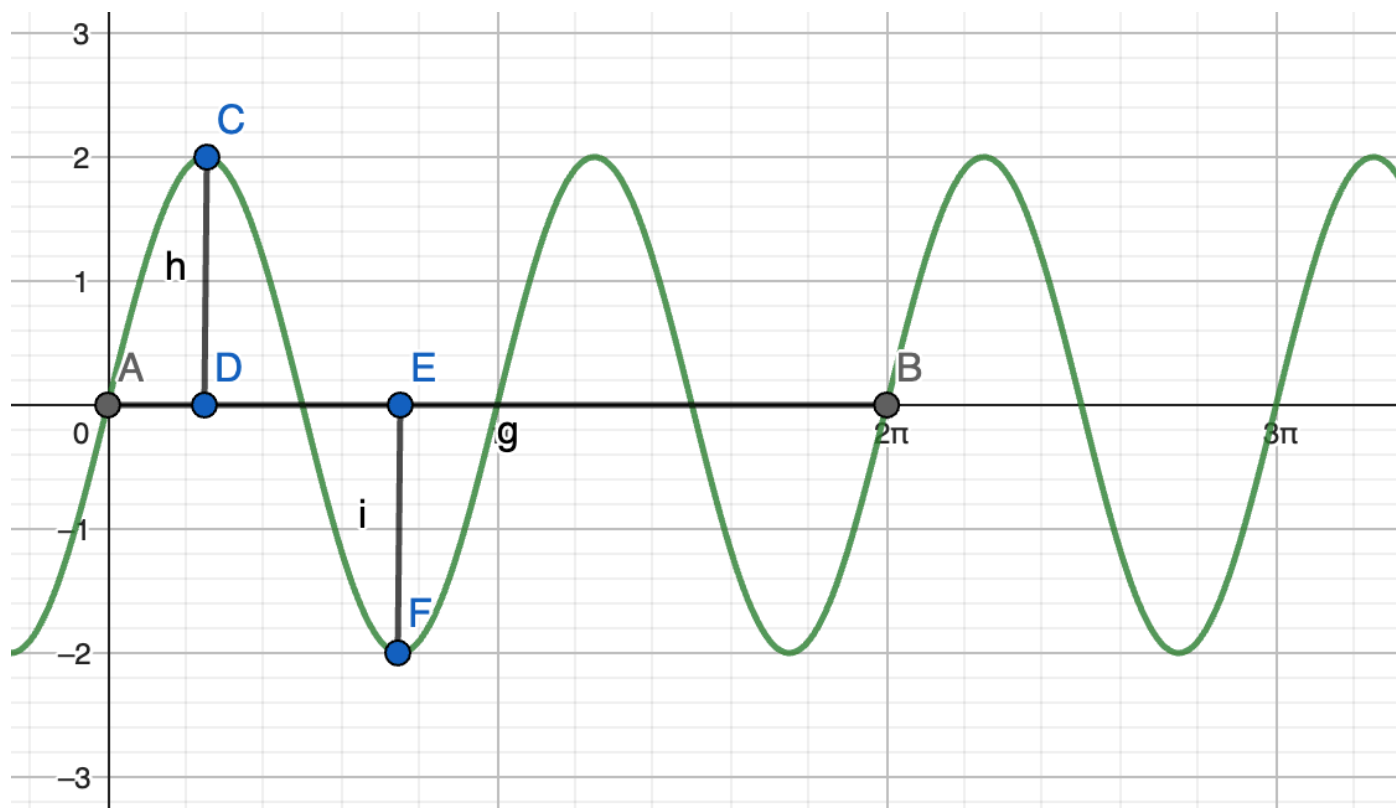
We first start by understanding the basics of waves, their representation, and terminology.

Here, we have plotted a graph of the function: $y = 2 \sin x$, where x is the angle in radians and y is the sine of that angle.

- Wavelength (AB): The length of a single whole oscillation / cycle. Here, 0 to 2π .
- Frequency: The number of wavelengths in 1 second. Measured in Hertz (Hz). Frequency controls PITCH. Higher the frequency, shorter the wavelength, higher the pitch. Lower the frequency, longer the wavelength, lower the pitch.
- Amplitude (CD or EF): The "height" of the wave, the number "2" in the above function definition. Amplitude controls LOUDNESS or INTENSITY. Higher the amplitude, louder the sound.



If we plot $y = 2 \sin(2x)$, we now see that the amplitude remains the same, but the frequency has doubled, and the wavelength has halved.



2. Fast Fourier Transform (FFT)

Remember in the last lecture, how we looked at combinations of functions? We will continue with the same idea and look at what a Fast Fourier Decomposition or Transform (FFT) is. Basically, we now look at the opposite problem. In the last lecture, we looked at two pure functions, and figured out what their combination sum function is. This time, we invert the problem: given a complicated wave function, how can we decompose it into its simple component parts? Let's watch the first few minutes of this video which explains it very visually:

<https://www.youtube.com/watch?v=spUNpyF58BY> ↗ (<https://www.youtube.com/watch?v=spUNpyF58BY>)



<https://www.youtube.com/watch?v=spUNpyF58BY>

3. p5.sound

3.1. Load an audio file and play it

We need a function called `preload()` where we load all of our media, including sound, images, etc. We can use the function `loadSound()` to specify the name and type of file we want to load. Here, I

have already uploaded the sound file and am specifying it by name. Make sure the file path is correct.

```
let audio;
let slider;

function preload() {
  audio = loadSound('Bach-Prelude_In_C_major.mp3');
}

function setup() {
  createCanvas(400, 400);
  audio.play();
  //audio.loop();
  //audio.stop();
  //audio.setVolume(0.5); //0 (no volume) to 1 (full volume)
  //createSlider(min, max, value, step)
  slider = createSlider(0,1,0.5,0.02);}

function draw() {
  background(220);
  audio.setVolume(slider.value());
}
```

We can now also pan to the left and right speakers using a similar technique.

```
let audio;
let slider;
let sliderPan;

function preload() {
  audio = loadSound('Bach-Prelude_In_C_major.mp3');
}

function setup() {
  createCanvas(400, 400);
  audio.play();
  //audio.loop();
  //audio.stop();
  //audio.setVolume(0.5); //0 (no volume) to 1 (full volume)
  slider = createSlider(0,1,0.5,0.02);
  sliderPan = createSlider(-1,1,0,0.01);
}

function draw() {
  background(220);
  audio.setVolume(slider.value());
  audio.pan(sliderPan.value());
}
```

Using a very similar method, you can also control for the rate at which your audio file plays:

<https://p5js.org/reference/p5.SoundFile/rate/>  [\(https://p5js.org/reference/p5.SoundFile/rate/\)](https://p5js.org/reference/p5.SoundFile/rate/)

3.2. Visualising amplitude

```
let audio;
let button;
let amp;

function preload(){
  audio = loadSound('Bach-Prelude_In_C_major.mp3');
}
```

```

function setup() {
  createCanvas(400, 400);
  button = createButton('PLAY');
  button.mousePressed(togglePlaying);
  amp = new p5.Amplitude();
}

function draw() {
  background(220);
  print(amp.getLevel());
  r = amp.getLevel()*width;
  fill('red');
  circle(200,200,r);
}

function togglePlaying(){
  if(!audio.isPlaying()) {
    audio.play();
    audio.setVolume(1.0);
    button.html("PAUSE");
  } else {
    audio.stop();
    button.html("PLAY");
  }
}

```

And we can now use some of the last lectures content and ideas, and visualise the form of the amplitude as it changes:

```

let audio;
let button;
let amp;
let ampArray = [];

function preload(){
  audio = loadSound('Bach-Prelude_In_C_major.mp3');
}

function setup() {
  createCanvas(400, 400);
  button = createButton('PLAY');
  button.mousePressed(togglePlaying);
  amp = new p5.Amplitude();
}

function draw() {
  background(0);
  let r = amp.getLevel();
  ampArray.push(r);
  stroke(255);
  noFill();
  beginShape();
  for (i=0; i<ampArray.length; i++){
    y = map(ampArray[i],0,1,height/2,0);
    //vertex(i,height/2);
    vertex(i,y);
  }
  endShape();

  if (ampArray.length > width) {
    ampArray.splice(0,1);
  }
}

function togglePlaying(){
  if(!audio.isPlaying()) {
    audio.play();
    audio.setVolume(1.0);
    button.html("PAUSE");
  } else {

```

```

    audio.stop();
    button.html("PLAY");
  }
}

```

3.3. Visualising the waveforms through FFT

The p5.js reference is here: <https://p5js.org/reference/p5.sound/p5.FFT/> 
[\(https://p5js.org/reference/p5.sound/p5.FFT/\)](https://p5js.org/reference/p5.sound/p5.FFT/)

What we will now do is use the FFT idea to take in a piece of audio, use the FFT to pick apart the pure frequencies, and obtain the amplitudes of these frequencies, and visualise them.

```

let audio;
let button;
let fft;
let ampArray = [];

function preload(){
  audio = loadSound('Bach-Prelude_In_C_major.mp3');
}

function setup() {
  createCanvas(512, 512);
  button = createButton('PLAY');
  button.mousePressed(togglePlaying);
  fft = new p5.FFT(0,256); //0 refers to smoothing, 256 refers to the number of frequency bin
  s.
}

function draw() {
  background(0);
  let ampArray = fft.analyze();
  stroke(255);
  noFill();
  for (i=0; i<ampArray.length; i++){
    y = map(ampArray[i],0,255,height,0);
    strokeWeight(2);
    fill(y,y*0.5,y*0.1);
    //line(i,height,i,y);
    //line(i+100, height, i+100, y);
    rect(i*10, y, 10, height - y);
  }

  if (ampArray.length > width) {
    ampArray.splice(0,1);
  }
}

function togglePlaying(){
  if(!audio.isPlaying()) {
    audio.play();
    audio.setVolume(1.0);
    button.html("PAUSE");
  } else {
    audio.stop();
    button.html("PLAY");
  }
}
}

```

You can try to put in different audio inputs to check which parts of the frequency spectrum dominate.

A really cool example of how the idea of FFT has been used in large scale public art:

https://www.youtube.com/watch?v=1_sGxX-6hf0  https://www.youtube.com/watch?v=1_sGxX-6hf0



https://www.youtube.com/watch?v=1_sGxX-6hf0

Copyright © The University of Sydney. Unless otherwise indicated, 3rd party material has been reproduced and communicated to you by or on behalf of the University of Sydney in accordance with section 113P of the Copyright Act 1968 (Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice.

Live streamed classes in this unit may be recorded to enable students to review the content. If you have concerns about this, please visit our [student guide](https://canvas.sydney.edu.au/courses/4901/pages/zoom) (<https://canvas.sydney.edu.au/courses/4901/pages/zoom>) and contact the unit coordinator.
[Privacy Statement](https://www.sydney.edu.au/privacy-statement.html) (<https://www.sydney.edu.au/privacy-statement.html>)